

# Only One Pattern in C# with Examples:

In this article, I am going to discuss How to Implement **Only One Pattern in C#** Asynchronous Programming with Examples. Please read our previous article where we discussed How to Implement [Retry Pattern in C#](#) Asynchronous Programming with Examples.

## Only One Pattern in C# Asynchronous Programming:

Sometimes we will have multiple tasks, and all the tasks give us the same information, and we only want to use the first one to finish and cancel the rest. For that, we can use a pattern (Only One Pattern) that uses the cancellation token. An example of this will be if we need to obtain information from different providers that work asynchronously. And when we get a response from one, we want to cancel the other tasks.

## Example to understand Only One Pattern in C#:

Let's see an example to understand Only One Pattern in C#. Please have a look at the following image. The following ProcessingName is an asynchronous method. This method takes two parameters i.e. name and cancellation token. Then here we delay the execution for a random period of time between 1 to 10 seconds. And finally, we return the name by appending the Hello word. This method is going to call multiple times and we don't know for which call it will delay the execution for how much time as the waiting time generates randomly.

```
public static async Task<string> ProcessingName(string name, CancellationToken token)
{
    //Creating Dynamic Waiting Time
    //The following statement will generate a number between 1 and 10 dynamically
    var WaitingTime = new Random().NextDouble() * 10 + 1;
    await Task.Delay(TimeSpan.FromSeconds(WaitingTime));

    string message = $"Hello {name}";
    return message;
}
```

Every request that we make to this method is going to wait for a random amount of seconds. That means we don't know which request is going to finish first.

## Creating the Only One Pattern in C#:

Now, what we want to do is the following.

I am going to invoke the ProcessingName method four times with four different parameters, but I only want the first result. And immediately after I get the first result, I want to cancel every other request. Please have a look at the following image which exactly does the same.

Here, first, we are initializing our cancellation token. Then I am creating a list of names to be processed by the ProcessingName method. Then we are creating the tasks by using LINQ and Lambda expressions by passing the name and the cancellation token. It will invoke the ProcessingName method by passing the name and cancellation token. Then we call the WhenAny method by passing the tasks. The WhenAny

method creates a task that will be completed when any of the supplied tasks have been completed. Next, we fetch the first completed content and then cancel the token, and finally print the content on the Console.

```
public static async void OnlyOnePattern()
{
    //Creating the Cancellation Token
    var CTS = new CancellationTokenSource();

    //Creating the list of names to process by the ProcessingName method
    List<string> names = new List<string>() { "Pranaya", "Anurag", "James", "Smith" };

    //Creating the tasks by passing the name and cancellation token using Linq
    //It will invoke the ProcessingName method by passing name and cancellation token
    var tasks = names.Select(x => ProcessingName(x, CTS.Token));

    var task = await Task.WhenAny(tasks);

    //Fetch the first completed result
    var content = await task;

    //Cancel the token
    CTS.Cancel();

    //Print the content
    Console.WriteLine($"\\n{content}");
}
```

Next, we just need to call the OnlyOnePattern method from inside the Main method. The complete code example is given below.

```
using System;
using System.Collections.Generic;
using System.Threading;
using System.Threading.Tasks;
using System.Linq;
namespace AsynchronousProgramming
{
    class Program
    {
        static void Main(string[] args)
        {
            OnlyOnePattern();
            Console.ReadKey();
        }
        public static async void OnlyOnePattern()
        {
            //Creating the Cancellation Token
            var CTS = new CancellationTokenSource();
            //Creating the list of names to process by the ProcessingName method
```

```

List<string> names = new List<string>() { "Pranaya", "Anurag", "James", "Smith" };
Console.WriteLine($"All Names");
foreach (var item in names)
{
    Console.Write($"{item} ");
}
//Creating the tasks by passing the name and cancellation token using CTS
//It will invoke the ProcessingName method by passing name and cancellation token
var tasks = names.Select(x => ProcessingName(x, CTS.Token));

var task = await Task.WhenAny(tasks);
//Fetch the first completed result
var content = await task;
//Cancel the token
CTS.Cancel();
//Print the content
Console.WriteLine($"\\n{content}");
}
public static async Task<string> ProcessingName(string name, Cancellation token)
{
    //Creating Dynamic Waiting Time
    //The following statement will generate a number between 1 and 10
    var WaitingTime = new Random().NextDouble() * 10 + 1;
    await Task.Delay(TimeSpan.FromSeconds(WaitingTime));
    string message = $"Hello {name}";
    return message;
}
}

```

I run the above code three times and I got the following result. In your case, the result might vary. If you are getting the same result then try multiple times and at some point in time you will get a different result.

```

All Names
Pranaya Anurag James Smith
Hello Smith

```

**Run1**

```

All Names
Pranaya Anurag James Smith
Hello James

```

**Run2**

```

All Names
Pranaya Anurag James Smith
Hello Smith

```

**Run3**

So, from the above output, you can see that the WhenAny method creates a task that will be completed as soon as when any of the supplied tasks have been completed, and then it will immediately cancel the rest of the tasks. This is called only one pattern in C# asynchronous programming.

## Generic Only One Pattern in C# Asynchronous Programming:

For a better understanding, please have a look at the following image.

```
//The Generic OnlyOne Pattern
//Here the parameter IEnumerable<Func<CancellationToken, Task<T>>> functions specifies
//a collection of method that takes CancellationToken as a parameter and returns a Task<T>
public static async Task<T> GenericOnlyOnePattern<T>(IEnumerable<Func<CancellationToken, Task<T>>> functions)
{
    //Creating local CancellationTokenSource
    var cancellationTokenSource = new CancellationTokenSource();

    //Invoking the function by passing the Cancellation Token
    //It will invoke the functions which is pointed by the Func Generic Delegate
    var tasks = functions.Select(function => function(cancellationTokenSource.Token));

    //Calling the WhenAny method by passing the list of tasks
    //It create a task that represents the completion of one of the supplied tasks.
    //The return task's Result is the task that completed.
    var task = await Task.WhenAny(tasks);

    //Cancel the token
    cancellationTokenSource.Cancel();

    //Return the content
    return await task;
}
```

### Explanation of the above Code:

1. **IEnumerable<Func<CancellationToken, Task<T>>> functions**: A Func is a generic delegate that points to a method that returns something. Now, our OneOne Pattern will take multiple tasks. So, the parameter of Our Generic OnlyOne Pattern is going to be an IEnumerable of Func which take Cancellation Token as the input parameter and returns a Task of T i.e. IEnumerable<Func<CancellationToken, Task<T>>> and here we called this parameter as functions. So, here the parameter IEnumerable<Func<CancellationToken, Task<T>>> functions specify a collection of methods that takes CancellationToken as a parameter and returns a Task<T>.
2. **var cancellationTokenSource = new CancellationTokenSource()**: Then we are creating a local CancellationTokenSource instance.
3. **var tasks = functions.Select(function => function(cancellationTokenSource.Token))**: Then we are invoking the function by passing the Cancellation Token. It will invoke the functions which are pointed by the Func Generic Delegate. Actually, at this point it will not invoke the methods, it will just create the list of tasks to be invoked when we call the WhenAll method.
4. **var task = await Task.WhenAny(tasks)**: Then we call the WhenAny method by passing the list of tasks. The WhenAny method creates a task that represents the completion of one of the supplied tasks. The return task's Result is the task that is completed.
5. **cancellationTokenSource.Cancel()**: Once we get the result from the WhenAny method i.e. once the WhenAny method is completed, then we need to cancel the token.
6. **return await task**: Returning the completed task result.

## How to use the Generic OnlyOne Pattern in C#?

We have created our generic Only One Pattern in C# Asynchronous Programming. Now, let us see how to use the Generic OnlyOne Pattern in C#. For this please have a look at the following image. Here, first, we are creating the collection of names to be processed by the ProcessName method. Remember, the Generic OnlyOne pattern accepts one parameter of IEnumerable<Func<Cancellation token, Task<T>>>, So, to call the Generic OnlyOne Pattern method we have created an IEnumerable of Func which should point to the ProcessName method by passing the name and cancellation token as a parameter using the LINQ select statement. And then we are calling the GenericOnlyOnePattern method and whatever the GenericOnlyOnePattern method return we print it on the Console window.

```
public static async void SomeMethod()
{
    //Creating the collection of names
    List<string> names = new List<string>() { "Pranaya", "Anurag", "James", "Smith" };

    //Creating the IEnumerable of Generic Func which points to ProcessName method
    //by passing the name and cancellation token
    var tasks = names.Select(name =>
    {
        Func<Cancellation token, Task<string>> func = (ct) => ProcessName(name, ct);
        return func;
    });

    //Calling the GenericOnlyOnePattern method by passing the collection of Func delegate
    var content = await GenericOnlyOnePattern(tasks);

    //Printing the content
    Console.WriteLine($"{content}");
}
```

Next, from the main method, we need to call the SomeMethod. The complete example is given below.

```
using System;
using System.Collections.Generic;
using System.Threading;
using System.Threading.Tasks;
using System.Linq;
namespace AsynchronousProgramming
{
    class Program
    {
        static void Main(string[] args)
        {
            SomeMethod();
            Console.ReadKey();
        }
        public static async void SomeMethod()
        {
            //Creating the collection of names
```

```

        List<string> names = new List<string>() { "Pranaya", "Anurag", "Jas",
"Smith" };
        Console.WriteLine($"All Names");
        foreach (var item in names)
        {
            Console.Write($"{item} ");
        }
        //Creating the IEnumerable of Generic Func which points to ProcessName
        //by passing the name and cancellation token
        var tasks = names.Select(name =>
        {
            Func<CancellationToken, Task<string>> func = (ct) => ProcessName(name, ct);
            return func;
        });
        //Calling the GenericOnlyOnePattern method by passing the collection of
delegate
        var content = await GenericOnlyOnePattern(tasks);
        //Printing the content
        Console.WriteLine($"\\n{content}");
    }
    //The Generic OnlyOne Pattern
    //Here the parameter IEnumerable<Func<CancellationToken, Task<T>>> functions
specify
    //a collection of method that takes Cancellation Token as a parameter and
a Task<T>
    public static async Task<T> GenericOnlyOnePattern<T>
    (IEnumerable<Func<CancellationToken, Task<T>>> functions)
    {
        //Creating local CancellationTokenSource
        var cancellationTokenSource = new CancellationTokenSource();

        //Invoking the function by passing the Cancellation Token
        //It will invoke the functions which is pointed by the Func Generic
        var tasks = functions.Select(function =>
function(cancellationTokenSource.Token));
        //Calling the WhenAny method by passing the list of tasks
        //It create a task that represents the completion of one of the specified
tasks.

        //The return task's Result is the task that completed.
        var task = await Task.WhenAny(tasks);
        //Cancel the token
        cancellationTokenSource.Cancel();
        //Return the content
        return await task;
    }
    public static async Task<string> ProcessName(string name, Cancellation
token)
    {
        //Creating Dynamic Waiting Time
        //The following statement will generate a number between 1 and 10

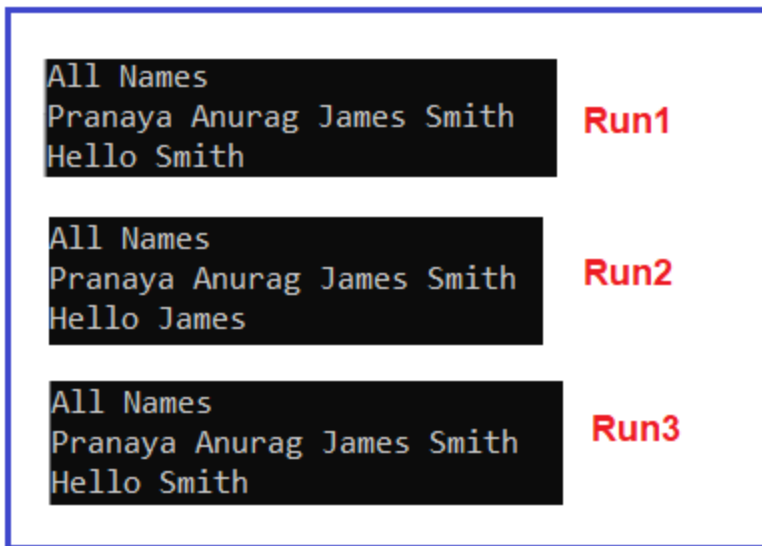
```

```

    var WaitingTime = new Random().NextDouble() * 10 + 1;
    await Task.Delay(TimeSpan.FromSeconds(WaitingTime));
    string message = $"Hello {name}";
    return message;
}
}
}

```

I run the above code three times and I got the following result. In your case, the result might vary. If you are getting the same result then try multiple times and at some point in time you will get a different result.



## OnlyOne Pattern with Different Methods in C#:

As of now, we are using our Only One Pattern to make the same operation over a collection. But We may not want that always. Maybe we have two different methods that we want to run at the same time, but we want to cancel one method after the other method finishes. This is also possible using Only One Pattern in C#.

First, create the following two methods that we are going to process using Only One Pattern. The code already we have explained. So, please go through the comment lines. The logic is going to be the same in both methods.

```

public static async Task<string> HelloMethod(string name, CancellationToken token)
{
    //Creating Dynamic Waiting Time
    //The following statement will generate a number between 1 and 10 dynamically
    var WaitingTime = new Random().NextDouble() * 10 + 1;
    await Task.Delay(TimeSpan.FromSeconds(WaitingTime));

    string message = $"Hello {name}";
    return message;
}

public static async Task<string> GoodbyeMethod(string name, CancellationToken token)
{
    //Creating Dynamic Waiting Time
    //The following statement will generate a number between 1 and 10 dynamically
    var WaitingTime = new Random().NextDouble() * 10 + 1;
    await Task.Delay(TimeSpan.FromSeconds(WaitingTime));

    string message = $"Goodbye {name}";
    return message;
}

```

Next, modify the GenericOnlyOnePattern pattern as shown in the below image. The body is going to be the same as the previous version of the GenericOnlyOnePattern pattern so I am not explaining the body. The only difference is the parameter. Here, we are using params array instead of IEnumerable. The rest of the things are going to be the same.

```

public static async Task<T> GenericOnlyOnePattern<T>(params Func<CancellationToken, Task<T>>[] functions)
{
    var cancellationTokenSource = new CancellationTokenSource();
    var tasks = functions.Select(function => function(cancellationTokenSource.Token));
    var task = await Task.WhenAny(tasks);

    cancellationTokenSource.Cancel();
    return await task;
}

```

Next, we need to use the above GenericOnlyOnePattern method. So, modify the SomeMethod as shown in the below image. As the GenericOnlyOnePattern takes the params array as an input parameter, so we can call different types of methods. Here, we are passing two different methods, and then whatever results they return we simply print on the console window.

```

public static async void SomeMethod()
{
    //Calling two Different Method using Generic Only One Pattern
    var content = await GenericOnlyOnePattern(
        //Calling the HelloMethod
        (ct) => HelloMethod("Pranaya", ct),
        //Calling the GoodbyeMethod
        (ct) => GoodbyeMethod("Anurag", ct)
    );

    //Printing the result on the Console
    Console.WriteLine($"{content}");
}

```

The complete example code is given below.

```

using System;
using System.Threading;
using System.Threading.Tasks;
using System.Linq;
namespace AsynchronousProgramming
{
    class Program
    {
        static void Main(string[] args)
        {
            SomeMethod();
            Console.ReadKey();
        }
        public static async void SomeMethod()
        {
            //Calling two Different Method using Generic Only One Pattern
            var content = await GenericOnlyOnePattern(
                //Calling the HelloMethod
                (ct) => HelloMethod("Pranaya", ct),
                //Calling the GoodbyeMethod
                (ct) => GoodbyeMethod("Anurag", ct)
            );
            //Printing the result on the Console
            Console.WriteLine($"{content}");
        }
        public static async Task<T> GenericOnlyOnePattern<T>(params
Func<CancellationToken, Task<T>>[] functions)
        {
            var cancellationTokenSource = new CancellationSource();
            var tasks = functions.Select(function =>
function(cancellationTokenSource.Token));
            var task = await Task.WhenAny(tasks);

```

```

        cancellationTokenSource.Cancel();
        return await task;
    }

    public static async Task<string> HelloMethod(string name, Cancellation
token)
    {
        var WaitingTime = new Random().NextDouble() * 10 + 1;
        await Task.Delay(TimeSpan.FromSeconds(WaitingTime));
        string message = $"Hello {name}";
        return message;
    }

    public static async Task<string> GoodbyeMethod(string name, Cancellati
token)
    {
        var WaitingTime = new Random().NextDouble() * 10 + 1;
        await Task.Delay(TimeSpan.FromSeconds(WaitingTime));
        string message = $"Goodbye {name}";
        return message;
    }
}

```

Now, run the above multiple time and you observe that sometimes the HelloMethod execute first and sometime GoodbyeMethod execute first. Once one method is completed the other method is canceled.

## WhenAny Methods of Task Class in C#:

The Task class in C# provides the following four overloaded versions of the WhenAny Method.

1. **WhenAny(IEnumerable<Task> tasks):** It creates a task that will complete when any of the supplied tasks have been completed. Here, the parameter tasks specify the tasks to wait on for completion. It returns a task that represents the completion of one of the supplied tasks. The return task's Result is the task that is completed.
2. **WhenAny<TResult>(IEnumerable<Task<TResult>> tasks):** It creates a task that will be complete when any of the supplied tasks have been completed. Here, the parameter tasks specify the tasks to wait on for completion. Here, the type parameter TResult specifies the type of the completed task. It returns a task that represents the completion of one of the supplied tasks. The return task's Result is the task that is completed.
3. **WhenAny(params Task[] tasks):** It creates a task that will complete when any of the supplied tasks have been completed. Here, the parameter tasks specify the tasks to wait on for completion. It returns a task that represents the completion of one of the supplied tasks. The return task's Result is the task that is completed.
4. **WhenAny<TResult>(params Task<TResult>[] tasks):** It creates a task that will complete when any of the supplied tasks have been completed. Here, the parameter tasks specify the tasks to wait on for completion. Here, the type parameter TResult specifies the type of the completed task. It returns a task that represents the completion of one of the supplied tasks. The return task's Result is the task that is completed.

In the next article, I am going to discuss [How to Control the Result of a Task in C#](#) with Examples. Here, in this article, I try to explain **Only One Pattern in C# Asynchronous Programming** with Examples. I hope you

enjoy this [Only One Pattern in C# Asynchronous Programming with Examples](#) article.